STATISTICS & MACHINE LEARNING · LESSON 29 OF 30

Recommendation Systems: Collaborative, Content-Based and Hybrid

Build recommenders that suggest the next product or item. Learn collaborative filtering from a user-item matrix, content-based filtering from item features, how to combine them, how to handle new users, and how to evaluate with precision at k and hit rate without recommending what people already own.

Gap-fill lesson | ✦ 2026 | Advanced | ⏱ 150 min

By the end of this lesson you can

- Explain collaborative, content-based and hybrid recommendation
- Build a user-item matrix and compute item-item similarity
- Recommend items a user has not yet bought, with a score
- Evaluate a recommender with precision at k and hit rate on held-out interactions
- Describe the cold-start problem and two ways to handle it

Datasets used

`retail_transactions.csv`

What a recommender does

A recommender answers one question: **which items should this person see next?** Streaming services suggest films, shops suggest products, banks suggest products, and job boards suggest roles. Good recommendations raise sales and engagement; bad ones show things people already own or never want.

CASE STUDY **The "customers also bought" widget**

An online shop wants a "you may also like" list on each product page. The data is a log of which customers bought which products. The task is to build a recommender, measure whether it suggests products people actually buy, and avoid suggesting what the customer already owns.

Step 1: the user-item matrix

Almost every recommender starts with a table: one row per user, one column per item, and a value showing interaction (a purchase, a rating, a click). Most cells are empty, because nobody buys everything.

Python

```python
import pandas as pd

# A tiny purchase log: 1 means the customer bought the product
purchases = pd.DataFrame(
    [("Asha", "tea"), ("Asha", "biscuits"), ("Asha", "honey"),
     ("Ravi", "tea"), ("Ravi", "biscuits"), ("Ravi", "jam"),
     ("Meera", "tea"), ("Meera", "honey"),
     ("Kabir", "coffee"), ("Kabir", "sugar"),
     ("Tara", "biscuits"), ("Tara", "jam")],
    columns=["user", "item"],
)
matrix = pd.crosstab(purchases["user"], purchases["item"]).clip(upper=1)
print(matrix)
```

OUTPUT

```
item   biscuits  coffee  honey  jam  sugar  tea
user
Asha          1       0      1    0      0    1
Kabir         0       1      0    0      1    0
Meera         0       0      1    0      0    1
Ravi          1       0      0    1      0    1
Tara          1       0      0    1      0    0
```

Each row is a basket. The matrix is the only input the collaborative method needs. Real logs are much larger and sparser, so the matrix is usually stored in a sparse format.

Step 2: item-item similarity (collaborative filtering)

Collaborative filtering says: items bought by the same kinds of customers are related. Measure how similar two items are by the customers who bought each one. **Cosine similarity** of two items' columns is a standard measure: 1 means they are bought by exactly the same customers, 0 means no overlap.

Python

```
import numpy as np

items = matrix.columns
vectors = matrix.T.values.astype(float)          # one row per item
norms = np.linalg.norm(vectors, axis=1, keepdims=True)
norms[norms == 0] = 1                            # avoid dividing by zero
similarity = pd.DataFrame(vectors @ vectors.T / (norms @ norms.T), index=items,
columns=items)
print(similarity.round(2))
```

OUTPUT

```
item      biscuits  coffee  honey   jam  sugar   tea
item
biscuits      1.00     0.0   0.41  0.82    0.0  0.67
coffee        0.00     1.0   0.00  0.00    1.0  0.00
honey         0.41     0.0   1.00  0.00    0.0  0.82
jam           0.82     0.0   0.00  1.00    0.0  0.41
sugar         0.00     1.0   0.00  0.00    1.0  0.00
tea           0.67     0.0   0.82  0.41    0.0  1.00
```

Tea and biscuits are bought together by Asha and Ravi, so their similarity is high. Coffee and sugar, which only Kabir bought, are similar only to each other.

Step 3: recommend what the user does not have

To recommend for a user, add up the similarity of every item to the items they already own, then **remove the items they already own**. Removing owned items is the step beginners forget, and it is the most visible failure in practice.

Python

```python
def recommend(user, matrix, similarity, k=3):
    owned = matrix.columns[matrix.loc[user] == 1]
    scores = similarity[owned].sum(axis=1)
    scores = scores.drop(labels=owned, errors="ignore")     # never recommend owned items
    return scores.sort_values(ascending=False).head(k)

print(recommend("Meera", matrix, similarity))
print(recommend("Ravi", matrix, similarity))
```

OUTPUT

```
item
biscuits    1.074915
jam         0.408248
coffee      0.000000
dtype: float64
item
honey     1.224745
coffee    0.000000
sugar     0.000000
dtype: float64
```

For Meera, the top suggestion is an item that similar customers buy with tea and honey, not tea or honey themselves. The score is only a ranking, not a probability.

Step 4: content-based filtering

Content-based filtering uses the features of each item rather than the behaviour of other users: category, brand, price, description. It recommends items that look like the ones the user liked. It works for a new item with no purchase history, because its features are known.

Python

```python
features = pd.DataFrame({
    "category": ["food", "food", "food", "food", "food", "beverage", "beverage"],
    "price_band": ["low", "low", "mid", "low", "low", "mid", "mid"],
}, index=["tea", "biscuits", "honey", "jam", "sugar", "coffee", "milk"])
encoded = pd.get_dummies(features).astype(float)

profile = encoded.loc[["tea", "honey"]].mean()          # the user's taste: average of liked items
candidates = encoded.drop(index=["tea", "honey"])
content_scores = candidates.dot(profile) / (np.linalg.norm(candidates, axis=1) * np.linalg.norm(profile))
print(content_scores.sort_values(ascending=False).round(2))
```

OUTPUT

```
biscuits    0.87
jam         0.87
sugar       0.87
coffee      0.29
milk        0.29
dtype: float64
```

Content-based suggestions are easy to explain ("similar to what you bought") but they can narrow tastes too much: the user only ever sees more of the same.

Step 5: hybrid systems and cold start

A **hybrid** recommender blends the two scores: `score = w × collaborative + (1 - w) × content`. The weight `w` is tuned on held-out data.

Cold start is the problem of a new user with no history or a new item with no purchases. Common responses:

Situation	Response
New user, no history	Show popular items in the category they came from, or ask for preferences at sign-up
New item, no purchases	Use content-based scores from its features until purchases arrive
New shop, little data	Start with editorial picks or bestsellers and collect data

INTERVIEW CORNER

How would you recommend products to a new user with no history?

Start with a popularity-based list, optionally filtered by the category or search term the user came from. Ask one or two preference questions at sign-up. As soon as the user has interactions, switch to the hybrid model. Measure the early experience separately, because the cold-start results are usually weaker.

Step 6: evaluation

Recommenders are judged on held-out interactions. Hide one or two purchases per user, recommend from what remains, and check whether the hidden items appear in the top k.

- **Precision at k**: of the k items recommended, the share that the user actually bought (on the held-out set).
- **Hit rate**: the share of users for whom at least one hidden item appears in the top k.

Python

```
def precision_at_k(recs, hidden, k):
    top = list(recs[:k])
    return len(set(top) & set(hidden)) / k

# Hide one purchase per customer, rebuild the matrix, recommend, and compare
held_out = {"Asha": "honey", "Ravi": "jam", "Meera": "honey", "Kabir": "sugar", "Tara":
"jam"}
train = purchases[~purchases.apply(lambda r: held_out.get(r["user"]) == r["item"], axis=1)]
train_matrix = pd.crosstab(train["user"],
train["item"]).clip(upper=1).reindex(columns=matrix.columns, fill_value=0)
vec = train_matrix.T.values.astype(float)
nrm = np.linalg.norm(vec, axis=1, keepdims=True); nrm[nrm == 0] = 1
sim_train = pd.DataFrame(vec @ vec.T / (nrm @ nrm.T), index=items, columns=items)

precisions, hits = [], 0
for user, hidden_item in held_out.items():
    recs = recommend(user, train_matrix, sim_train, k=3).index.tolist()
    precisions.append(precision_at_k(recs, [hidden_item], k=3))
    hits += hidden_item in recs
print("precision@3:", round(float(np.mean(precisions)), 3), " hit rate:", round(hits /
len(held_out), 3))
```

OUTPUT

```
precision@3: 0.2  hit rate: 0.6
```

With only 5 users, these numbers are noisy and not a result to report. The point is the method: a real evaluation uses thousands of users, several splits, and a baseline such as "most popular items" to beat.

::::warn Do not judge by accuracy alone Most items are never bought by a given user, so a model that recommends bestsellers can score well on accuracy without being useful. Compare against the popularity baseline, and check diversity (are the suggestions varied?) and whether recommendations are repeated for everyone. ::::

::::tip Choose the simplest method that beats the baseline Start with popularity. Add item-item similarity when you have enough purchases. Add content features for new items. Only add more complex models (matrix factorisation, neural recommenders, libraries such as Surprise) when the simpler versions have a measured gap. ::::

::::key What to remember - A recommender starts with a user-item matrix. - Collaborative filtering uses patterns across users; content-based uses item features; hybrid blends them. - Always exclude items the user already owns. - Cold start needs a plan: popularity, preference questions and content features. - Evaluate on held-out interactions with precision at k and hit rate, against a popularity baseline. ::::

Quick quiz

5 questions · tap an answer to check it

1. What does collaborative filtering rely on?

A Item descriptions only

B Patterns of which users interact with which items

C The user's age

D Random ratings

Answer: B. It uses the behaviour of many users to find related items.

2. **Cosine similarity between two items is 1. This means:**

A The items are priced the same

B They are bought by exactly the same customers (in proportion)

C One item is more popular

D They should never be recommended together

Answer: B. A value of 1 means identical direction in the user-item vectors.

3. **Why must owned items be removed from a recommendation list?**

A They cannot be scored

B Recommending what the customer already has is useless and easy to notice

C It makes the list longer

D The model requires it

Answer: B. Removing owned items is a basic rule of the product.

4. **A new product has no purchases yet. Which approach can still rank it?**

A Collaborative filtering alone

B Content-based filtering using its features

C Precision at k

D Hit rate

Answer: B. Content features exist before any interaction.

5. Precision at 5 is 0.4. What does that mean?

A 40% of users were recommended something

B 2 of the 5 recommended items were in the user's held-out purchases, on average

C The model is 40% accurate on all items

D 5 items were recommended 40 times

Answer: B. Precision at k is the share of the top k that are relevant.

EXERCISE **Recommender with a baseline**

Using a ratings or purchase file you have (or the retail transactions dataset), build an item-item recommender with the functions in this lesson. Hold out one item per user, compare precision at 5 and hit rate against a popularity baseline (recommend the five most-bought items the user does not own), and write three sentences on whether the recommender is worth the extra complexity.

Show solution (worked answer)

Hints: build the matrix from a 12-month window, keep only users with at least three items so each has a training history, and compute the popularity list from the training matrix only. Report both numbers side by side.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.